

Module Summary — AI Programming Foundations Project

Name: Brian Paul Roche **Programme:** MSc Artificial Intelligence — Capstone, Project 1 **Date:** 22 August 2026 **Dataset:** Titanic passenger manifest (891 rows, 12 columns)

Overview

This project builds a reusable, reproducible Python data workflow on the Titanic passenger manifest: ingestion with pandas, cleaning through two documented functions, exploration through a single reusable analysis function, and three labelled visualisations. The dataset is the 891-row Titanic manifest published at <https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv>. It was selected because its missing data is structurally varied rather than uniform, which forces the cleaning stage to be a set of reasoned decisions rather than a single blanket operation.

Dataset Description

The dataset records 891 passengers across 12 variables, comprising a binary survival outcome, passenger class, name, sex, age, counts of siblings/spouses and parents/children aboard, ticket number, fare paid, cabin, and port of embarkation. Overall survival in the sample is 38.4%.

Three columns carry missing values, and each is missing in a different way and to a different degree: Age for 177 passengers (19.9%), Cabin for 687 (77.1%), and Embarked for 2 (0.2%). The analysis focuses on survival as the outcome, with passenger class, sex, age, family composition and the presence or absence of a cabin record as the explanatory variables.

Workflow Description

Ingestion. The CSV is loaded with `pandas.read_csv`, and shape and per-column missingness are printed immediately, before any transformation. Establishing the missing-data profile first is what makes the subsequent cleaning decisions defensible rather than reflexive.

Cleaning. Two functions, each with a docstring stating what it does and why. Both return new DataFrames rather than mutating their input, so that re-running any cell mid-notebook cannot silently compound a transformation — a property Danchev (2022) identifies as central to a notebook-based workflow being genuinely reproducible rather than merely re-executable.

Exploratory analysis. A single `explore()` function returns a numeric summary, survival rates broken down by each categorical variable with group counts, and a correlation matrix. Writing it as a function rather than as inline cells means the entire summary can be regenerated unchanged after any modification to the cleaning stage.

Visualisations. Three figures, each with a title and labelled axes: survival rate by class and sex, age distribution by outcome, and the correlation matrix of numeric features.

Summary. A written interpretation covering findings, patterns, limitations and assumptions.

Key Decisions and Assumptions

Age was imputed within groups, not globally. Age distribution differs materially by passenger class and sex, so 177 missing values were filled with the median of each passenger's own `Pclass` and `Sex` group. A single global median would have flattened real variation and pulled every imputed passenger toward one central value.

Cabin was neither imputed nor simply deleted. At 77.1% missing, imputation would have been fabrication. But the missingness was not random: passengers with a recorded cabin survived at 66.7% ($n=204$) against 30.0% for those without ($n=687$). The column was therefore replaced with a binary `HasCabin` flag and the original dropped — preserving the signal carried by the *fact* of the record while inventing nothing. This is the single most consequential decision in the project, and it is a direct application of the principle that raw data should be treated as read-only and all transformations recorded explicitly rather than performed silently (Wilson et al., 2017).

Embarked was filled with the mode. Two rows, immaterial at that volume.

Features were derived rather than used raw. `SibSp` and `Parch` are only meaningful in combination, so `FamilySize` and `IsAlone` were derived from them. `Title` was extracted from `Name`, collapsing to five categories, because it encodes age, sex and social standing in one low-cardinality variable.

Identifiers were dropped. `PassengerId`, `Ticket` and `Name` carry no direct analytical content once `Title` has been extracted.

EDA focus. Survival rate by group was chosen over distributional comparison because the outcome is binary and group rates are directly interpretable.

What each plot was designed to show. Figure 1 tests whether class and sex act independently. Figure 2 tests whether age separates the outcomes at all, and simultaneously exposes the distortion introduced by imputation. Figure 3 tests for redundancy among the numeric features ahead of any future modelling.

Results and Interpretation

Survival was governed far more by who a passenger was than by anything else recorded.

Sex is the strongest single variable: 74.2% of women survived against 18.9% of men. Passenger class is next: 63.0% in first, 47.3% in second, 24.2% in third.

Figure 1 shows these do not act independently — female survival exceeds 90% in first and second class but falls to roughly half in third, while male survival is low everywhere and lowest in third. The combination spans nearly the full range of the outcome variable.

Title, derived from Name, reproduces and refines this: Mrs 79.2%, Miss 69.8%, Master 57.5%, Other 44.4%, Mr 15.7%. The Master category — boys — surviving at 57.5% against 15.7% for adult men indicates that age mattered chiefly at the youngest extreme.

Figure 2 confirms this. The age distributions of survivors and non-survivors overlap heavily, and age alone separates the outcomes poorly. Only the youngest band shows survivors in the majority. The concentration of mass in the mid-twenties is partly an artefact of imputation and is visible in the plot rather than concealed by it.

Family composition matters at a smaller scale: passengers travelling with family survived at 50.6% against 30.4% for those alone.

The HasCabin flag — derived from the column that was 77% empty — separates the outcomes more sharply than age does: 66.7% against 30.0%.

Figure 3 shows no strongly correlated numeric pair, meaning little redundancy to remove before modelling. Fare and HasCabin show the clearest positive relationship, consistent with both proxying wealth. SibSp and Parch correlate with FamilySize by construction, so only the derived variable should be carried forward.

Responsible Practice — Bias and Data Quality

Three cleaning choices in this dataset could have introduced bias, and each was handled explicitly.

Listwise deletion would have been the most damaging option. Dropping every row with a missing value would have removed 708 of 891 passengers — and not at random. Third-class passengers are far likelier to lack a cabin record, so the surviving sample would have been disproportionately wealthy and every survival rate computed from it biased upward in a predictable direction.

Global imputation would have manufactured false homogeneity. Filling all 177 missing ages from one median would have suppressed genuine between-group variation. Group-wise imputation reduces but does not eliminate the problem, and the residual distortion is stated openly in the interpretation of

Figure 2 rather than left for a reader to discover.

Discarding a mostly-empty column would have destroyed real signal.

The HasCabin result demonstrates that missingness itself was informative. Deleting Cabin on a missingness threshold alone — a common automated default — would have lost one of the strongest relationships in the dataset.

The general principle applied throughout is that the raw file is treated as read-only and every transformation is expressed in code that can be inspected and re-run, rather than applied by hand (Wilson et al., 2017). What would be done next is to test the sensitivity of the reported survival rates to the imputation strategy, by re-running the analysis with ages left missing and with a global median, and reporting the spread.

Reproducibility

The work can be rerun by any third party from the repository alone. `requirements.txt`, generated with `pip freeze`, fixes the package environment. `data_workflow.ipynb` runs top to bottom without manual intervention, provided `titanic.csv` is present in the same directory; the dataset link is recorded in both the notebook and the README so it can be re-obtained. The repository carries multiple commits and a separate development branch, so the sequence in which the analysis was built is inspectable rather than presented as a single finished artefact.

Danchev (2022) makes the case that reproducibility in a Jupyter-based workflow depends on the notebook being executable end to end in a clean environment rather than merely on the author's machine, and on the analytical steps being expressed as code rather than as manual manipulation. Both conditions are met here: the cleaning and exploration stages are functions, not ad-hoc cells, and the notebook was executed from a clean kernel before submission.

References

Danchev, V. (2022). Reproducible Data Science with Python: An Open Learning Resource. *Journal of Open Source Education*, 5(56), 156. <https://doi.org/10.21105/jose.00156>

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>